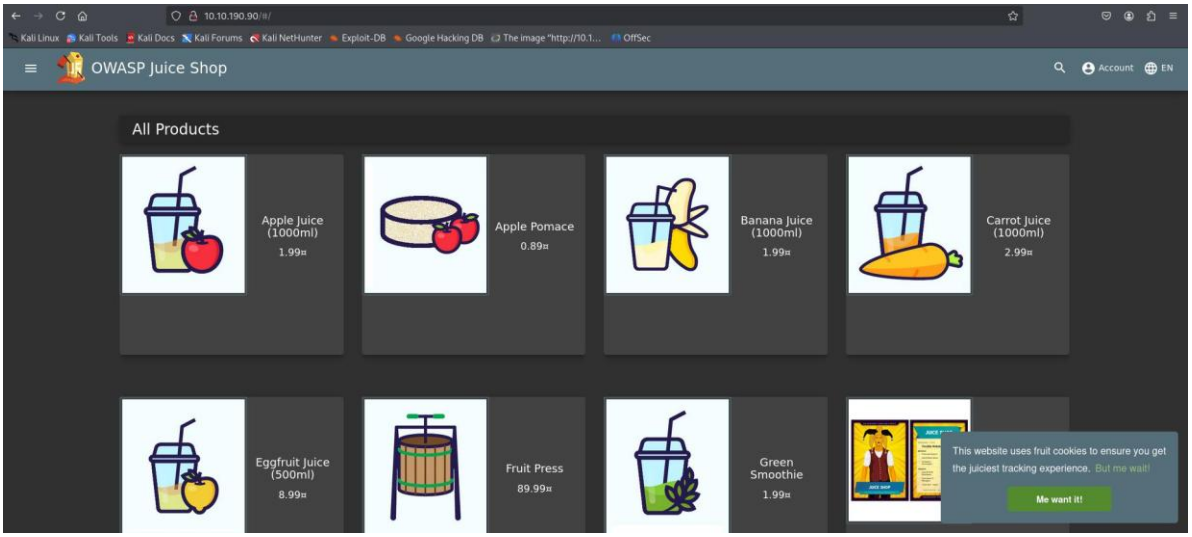


Student Name: -Kumbhar Dilip

College Name: - Indus University

Internship: - Cyber Security Internship

Organization: - DG Intern Hub

Week 8 Assignment: - Advanced VAPT – Attack Chain & Full Exploitation Project

SID: - dg/September/cyber/264

Mail: - dilipkumbhar.dginternshub@gmail.com

Task 8: - Theory + Practical

Page No.	Content
1	Introduction
2	Target & Scope
3	Attack Surface Mapping – Asset Discovery
4	Attack Surface Mapping – Services and Technologies
5	Attack Chain – Reconnaissance and Vulnerability Identification
6	Attack Chain – Exploitation and Validation
7	POC – Vulnerability Details and Reproduction Steps
8	POC – Evidence and Technical Validation
9	Risk & Impact Analysis – Severity and CVSS
10	Risk & Impact Analysis – Business Impact
11	Remediation – Immediate Fixes
12	Remediation – Security Hardening and Prevention
13	Conclusion
14	Appendix – Tools and References
15	Appendix – Supporting Evidence and Glossary

1. Introduction

1.1 What is Advanced VAPT?

Vulnerability Assessment and Penetration Testing (VAPT) is a security testing process used to identify weaknesses in applications, networks, and systems.

Advanced VAPT goes beyond finding individual vulnerabilities. It examines how multiple weaknesses can be combined to create a serious security risk.

For example, a weak access-control mechanism combined with improper session validation may allow an attacker to access information that should remain private.

1.2 Objective

The main objectives of this project are:

- Identify the application's entry points and attack surface.
- Test business logic and access-control weaknesses.
- Analyze HTTP requests and responses using Burp Suite.
- Develop an evidence-based attack-chain scenario.
- Understand the technical and business impact of vulnerabilities.
- Recommend security improvements and verify the fixes.

1.3 Importance of Attack Chain Testing

Testing individual vulnerabilities may not reveal the full risk to an application. Attack-chain testing helps security professionals understand how separate weaknesses can work together.

It helps organizations identify security gaps, protect sensitive information, and strengthen application security.

2. Target and Scope

2.1 Target Application

OWASP Juice Shop is an intentionally vulnerable web application designed for learning and practicing web application security testing.

It contains different types of security weaknesses that help learners understand common real-world application security problems.

2.2 Testing Scope

The assessment focuses on the following areas:

- Login and registration functionality.

- User profiles and access permissions.
- Input fields and application parameters.
- API endpoints and HTTP requests.
- Session and authentication handling.
- Business logic and authorization controls.

2.3 Tools Used

Tool	Purpose
OWASP Juice Shop	Vulnerable application for testing
Burp Suite Community Edition	Intercepting and analyzing HTTP requests
Web Browser	Accessing and testing application functions
Browser Developer Tools	Inspecting page behavior and application requests
OWASP ZAP (optional)	Additional web security analysis

2.4 Scope Limitations

Testing is restricted to the authorized lab environment. Only designated test accounts and data are used.

Production systems, real user accounts, and unrelated services are excluded from this assessment.

3. Attack Surface Mapping

3.1 What is Attack Surface Mapping?

Attack surface mapping is the process of identifying the places where a user or an attacker could interact with an application.

These entry points may include login forms, search fields, APIs, profile pages, and session-handling mechanisms.

3.2 Identified Entry Points

Entry Point	Testing Focus
Login page	Authentication and validation
Registration page	Input validation and account creation
User profile	Access control and data privacy
Search and input fields	Parameter handling

API endpoints	Request validation and authorization
Session cookies or tokens	Session security
Restricted pages	Authorization and direct access

3.3 Application Flow

The general application flow is:

1. The user opens the application.
2. The user registers or logs in.
3. The application creates an authenticated session.
4. The user accesses available features and resources.
5. The server processes requests and checks permissions.
6. The application returns a response.

Security testing checks whether the server correctly validates each request and prevents unauthorized access.

3.4 Mapping with Burp Suite

Burp Suite Proxy can capture requests and responses between the browser and the application.

The HTTP history helps identify endpoints, request methods, parameters, and response codes. The Target site map helps organize discovered application paths.

Evidence to include: Screenshot of the application and Burp Suite HTTP history.

4. Advanced Vulnerability Testing

4.1 Business Logic Flaws

Business logic flaws occur when an application does not correctly enforce its intended rules or workflow.

For example, an application may allow a user to skip a required step or submit an action without meeting the necessary conditions.

Testing approach:

1. Understand the normal application workflow.
2. Capture the relevant request in Burp Suite.
3. Test permitted variations using a lab account.
4. Compare the expected and actual behavior.

5. Record any confirmed security impact.

Recommended fix: Enforce business rules on the server side rather than trusting the browser.

4.2 Parameter Tampering

Parameter tampering occurs when a user changes values in an HTTP request to test whether the application validates them correctly.

For example, a request may contain a product quantity, resource identifier, or account-related parameter.

Testing approach:

1. Capture the original request.
2. Send it to Burp Repeater.
3. Modify one permitted test parameter.
4. Send the request again.
5. Compare the responses and verify whether the server enforces the correct rules.

Recommended fix: Validate every parameter on the server and enforce authorization before performing sensitive actions.

4.3 Insecure Direct Object Reference (IDOR)

IDOR is an access-control weakness that occurs when an application allows a user to access another user's resource by manipulating an object identifier without proper authorization checks.

For example, a profile request may include a user or resource ID. Changing that ID should not allow unauthorized access to another account's private information.

Testing approach:

1. Log in using a test account.
2. Capture a request for a resource belonging to that account.
3. Inspect the object identifier.
4. Where authorized, test access using a second test account that you control.
5. Check whether the server correctly denies access to resources that the requesting user does not own.

Recommended fix: Check the logged-in user's permissions for every requested resource on the server side.

4.4 Session Security

Session security ensures that only the correct authenticated user can access protected features.

Testing may examine cookie attributes, session expiration, logout behavior, and whether old sessions remain valid after security-sensitive changes.

Recommended fix: Use secure session handling, appropriate cookie attributes, session expiration, and session invalidation after logout or other sensitive events.

5. Attack Chain Explanation

5.1 What is an Attack Chain?

An attack chain is a sequence of security weaknesses or actions that may allow an attacker to achieve a larger objective.

The main purpose of this project is to understand how individual weaknesses could combine to create a more serious impact.

5.2 Example Attack Scenario

Scenario: Testing whether an access-control weakness could expose another test user's private information.

Step 1: Identify the entry point

Log in to the authorized Juice Shop lab using a test account. Explore the application and identify relevant requests.

Step 2: Capture the request

Use Burp Suite Proxy to capture a request that accesses a resource belonging to the logged-in test account.

Step 3: Analyze the request

Inspect the HTTP method, URL, parameters, and response. Identify whether the request contains a resource identifier.

Step 4: Test access control

Send the request to Burp Repeater. Using only authorized test accounts and resources, check whether changing the identifier causes the application to return data that the requesting account is not permitted to access.

Step 5: Verify the result

Compare the response with the expected authorization behaviour.

If the application denies access, record the result as a successful security control. If unauthorized access is demonstrated, capture the evidence and document the confirmed weakness.

Step 6: Analyze the impact

Determine what information or functionality could be exposed. Assess whether another verified weakness could increase the impact.

Step 7: Recommend fixes

Recommend server-side authorization checks and retest the affected endpoint after remediation.

Important: This is a proposed testing scenario. It must not be described as a successful attack unless your own lab results confirm it. A single IDOR finding is not automatically a multi-vulnerability attack chain.

5.3 Attack Flow Diagram

Application Discovery → Request Analysis → Vulnerability Testing → Access-Control Verification → Impact Assessment → Remediation and Retesting

The objective is to identify the complete risk while remaining within the authorized testing scope.

6. Proof of Concept (PoC)

6.1 What is a PoC?

A Proof of Concept (PoC) is evidence that demonstrates whether a suspected vulnerability is real.

A good PoC includes the testing steps, original request, modified request where applicable, server response, and an explanation of the result.

6.2 PoC: Access-Control Testing

Finding name: Potential Broken Object-Level Authorization (IDOR)

Target: Authorized OWASP Juice Shop lab

Tool: Burp Suite Community Edition — Proxy and Repeater

Procedure:

1. Log in using your own test account.
2. Capture a request for one of your own resources.
3. Send the request to Repeater.
4. Examine the resource identifier and response.

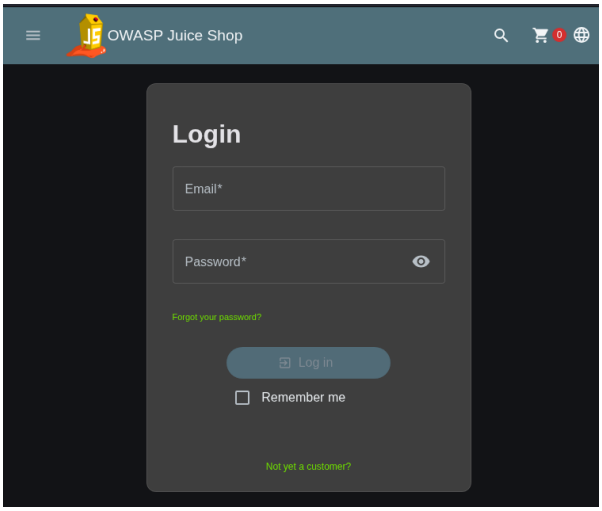
5. Test access to a second test account's resource only where permitted.
6. Record whether access is correctly denied or improperly allowed.

Evidence table

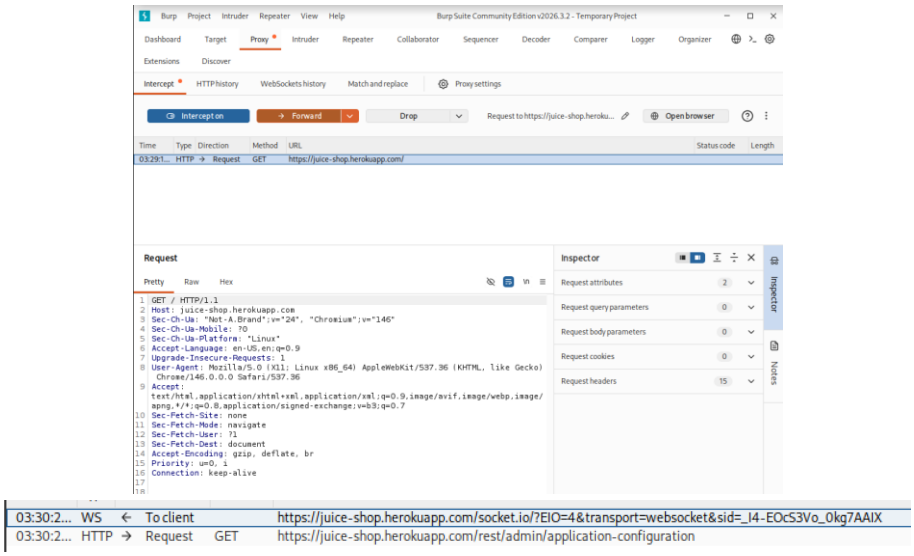
Test	Expected Result	Actual Result
Access own resource	Access permitted according to permissions	Record your result
Access another test account's private resource	Access denied	Record your result
Repeat test after remediation	Unauthorized access remains blocked	Record your result

Screenshots to add

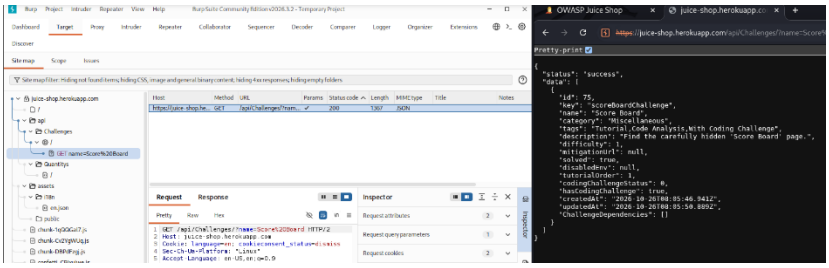
- **Figure 1:** Juice Shop application and testing environment.



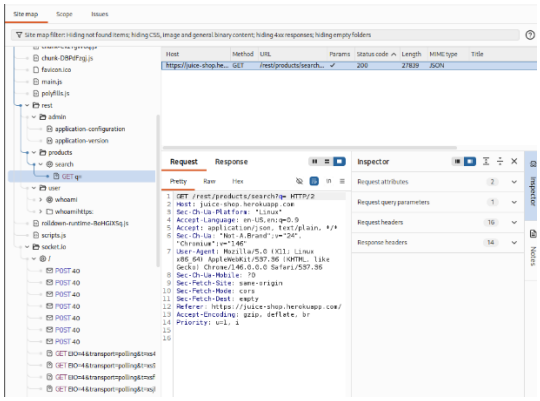
- **Figure 2:** Original HTTP request in Burp Suite.



- **Figure 3:** Repeater request and response during testing.



- **Figure 4:** Evidence of the observed authorization result.



- **Figure 5:** Retest results after applying the fix.

Do not include real passwords, session tokens, cookies, or other sensitive data in screenshots.

6.3 PoC Result

Status: Pending verification.

Complete this section after testing:

- **Expected behavior:** The application should enforce resource-level permissions.
- **Observed behavior:** Record the actual response from your lab.
- **Conclusion:** State whether the vulnerability was confirmed, not confirmed, or inconclusive.

Do not invent HTTP responses or claim successful exploitation without evidence.

7. Risk and Impact Analysis

7.1 Risk Assessment

Risk depends on the likelihood of exploitation and the potential impact on the application, users, and organization.

A confirmed vulnerability may be rated Low, Medium, High, or Critical based on factors such as exploitability, required privileges, exposed information, and business impact.

7.2 Potential Impact

Security Issue	Possible Impact
Broken access control	Unauthorized access to protected resources
Business logic flaw	Bypassing intended application rules
Parameter tampering	Unauthorized changes to application data or actions
Session weakness	Possible misuse of an authenticated session
Multiple weaknesses combined	Greater impact than an isolated issue

These are potential consequences, not confirmed outcomes of this assessment.

7.3 Business Impact

If successfully exploited, serious application vulnerabilities may result in:

- Exposure of confidential information.
- Unauthorized account or resource access.
- Loss of customer trust.
- Financial or operational damage.
- Legal and compliance concerns.

7.4 Severity

Assign severity only after reviewing the evidence and impact.

For example, an authorization weakness exposing sensitive information across accounts may be rated High depending on the data and circumstances. A minor issue with limited consequences may be rated Low or Medium.

The final severity for each finding must be supported by the actual test results.

8. Remediation Strategy

8.1 Fixing Business Logic Flaws

- Enforce workflow rules on the server.
- Validate the conditions required for each sensitive action.
- Do not rely on hidden buttons or browser-side restrictions.
- Test unusual and out-of-order requests.

8.2 Fixing Parameter Tampering

- Validate input types, values, and permitted ranges.
- Reject unexpected or unauthorized parameter changes.
- Enforce sensitive operations on the server.
- Apply appropriate validation to every API request.

8.3 Fixing IDOR

- Verify resource ownership and authorization on every request.
- Do not trust user-supplied resource identifiers.
- Apply access-control checks consistently across API endpoints.
- Return an appropriate denial response for unauthorized requests.
- Add automated tests for access-control rules.

8.4 Improving Session Security

- Use HTTPS in deployed applications.
- Set appropriate Secure, HttpOnly, and SameSite cookie attributes.
- Expire inactive sessions according to the application's security requirements.
- Invalidate sessions after logout and relevant security-sensitive events.
- Avoid exposing session tokens in URLs or logs.

8.5 Breaking the Attack Chain

Fixing one weakness may not be enough if other weaknesses remain.

To break the chain completely:

1. Correct the root cause of each confirmed vulnerability.
2. Enforce authentication and authorization on the server.
3. Validate all sensitive actions and input parameters.
4. Improve session management.
5. Retest every affected endpoint.
6. Verify that the combined attack scenario no longer succeeds.
7. Document the final results.

8.6 Before-and-After Verification

Test Area	Before Fix	After Fix
Resource authorization	Record the observed behaviour	Unauthorized access should be denied
Parameter validation	Record the observed behaviour	Invalid or unauthorized values should be rejected
Session handling	Record the observed behaviour	Sessions should follow the intended security rules

Populate the table with your actual results. A fix should be considered verified only after retesting confirms the expected behaviour.

9. Conclusion

This Advanced VAPT project focuses on understanding how multiple web application weaknesses may combine to create a larger security risk.

By using OWASP Juice Shop and Burp Suite, the assessment covers attack surface mapping, business logic testing, parameter tampering, access-control verification, session security, and Proof of Concept documentation.

The project also highlights the importance of assessing business impact, assigning evidence-based severity, and recommending practical security improvements.

The key learning is that finding a vulnerability is only one part of security testing. A complete assessment must explain the weakness, demonstrate its impact safely, recommend a fix, and verify that the issue has been resolved.

Through this project, I aim to improve my practical web application security testing skills and develop a better understanding of how professional VAPT assessments are performed.

10. References

1. OWASP Juice Shop — https://owasp.org/www-project-juice-shop/
2. OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
3. OWASP API Security Project — https://owasp.org/www-project-api-security/
4. PortSwigger Web Security Academy — https://portswigger.net/web-security
5. PortSwigger Burp Suite Documentation — https://portswigger.net/burp/documentation

---